Практическая работа №19
Настройка интерфейса приложения

1 Цель работы

1.1 Изучить основные элементы графического интерфейса Avalonia UI.

2 Литература

2.1 Avalonia documentation – Текст : электронный // AvaloniaUI, 2024. – URL: https://docs.avaloniaui.net/

3 Подготовка к работе

3.1 Повторить теоретический материал (см.п.2).

3.2 Изучить описание лабораторной работы.

4 Основное оборудование

4.1 Персональный компьютер.

5 Задание

5.1 Создайте новый проект Avalonia

5.1.1 Создать новый проект типа Avalonia C# Project и назвать его HelloApp

При создании:

- выбрать платформу Desktop, Android,
- указать фреймворк CommunityToolkit

5.1.2 Изменить запускаемый проект на HelloApp.Desktop. Запустить приложение.

5.1.3 Изменить запускаемый проект на HelloApp.Android. Запустить приложение.

5.2 Добавление страниц

5.2.1 Добавьте в приложение новую страницу HomeView (Добавить – Создать новый элемент – Avalonia – Avalonia User Control)

5.2.2 Измените пространство имен HomeView на HelloApp.Views (измените .axaml и .axaml.cs файлы)

5.2.3 На странице расположите TextBlock с текстом «Hello Avalonia!»

5.2.4 В MainWindow.axaml измените запускаемую страницу на HomeView. Запустите приложение.

5.2.5 Измените параметры TextBlock. Выровняйте его по центру экрана, измените цвет текста на зеленый, размер шрифта – 40

5.3 Взаимодействие разметки с кодом

5.3.1 В HomeView добавьте StackPanel и переместите TextBlock в нее.

5.3.2 Добавьте в StackPanel кнопку «Изменить текст»

5.3.3 Создать в папке ViewModels новый класс HomeViewModel. Данный класс должен быть наследником класса ViewModelBase и иметь модификатор

partial.

5.3.4 В классе определить поле с атрибутом ObservableProperty и метод с атрибутом RelayCommand:

```
[ObservableProperty]
private string text;

[RelayCommand]
public void ChangeText(){
    Text = "Новый текст"
}
```

5.3.5 Изучить описание данных атрибутов. Пересобрать проект

5.3.6 Откройте файл HomeView.axaml.cs и в конструкторе установите значение DataContext:

```
public HomeView()
{
     InitializeComponent();
     DataContext = new HomeViewModel();
}
```

5.3.7 В разметку UserControl страницы добавьте строки

```
xmlns:vm="using:HelloApp.ViewModels"
x:DataType="vm:HomeViewModel"
```

5.3.8 На странице привяжите значение Text элемента TextBlock к соответствующему свойству HomeViewModel и значение Command элемента Button к команде ChangeTextCommand.

```
<TextBlock Text="{Binding Text}"  ... />
<Button Command="{Binding ChangeTextCommand}" ... />
```

5.4 Контейнеры элементов управления

5.4.1 Добавить на экран DockPanel, задать для нее размеры.

5.4.2 В панели расположить 5 Rectangle разных цветов. Поочередно для них указать свойства:

- DockPanel.Dock=”Top” Height=”100”
- DockPanel.Dock=”Left” Width=”100”
- DockPanel.Dock=”Bottom” Height=”100”
- DockPanel.Dock=”Right” Width=”100”

Для последнего Rectangle свойство не устанавливайте.

5.4.3 Добавить на экран Grid, задать для него размеры и параметры Grid.ColumnDefinitions=”*,2*”, Grid.RowDefinitions=”Auto,100”

5.4.4 В сетке расположить 4 Rectangle разных цветов, расположив их в разных ячейках Grid при помощи Grid.Column и Grid.Row

5.4.5 Добавить на экран две StackPanel, для одной указать Orientation=”Horizontal”, для другой Orientation=”Vertical”.

5.4.6 В панелях расположить по 4 TextBlock с текстом, указать им различный размер.

6 Порядок выполнения работы

6.1 Выполнить все задания из п.5.

6.2 Ответить на контрольные вопросы.

7 Содержание отчета

7.1 Титульный лист

7.2 Цель работы

7.3 Ответы на контрольные вопросы

7.4 Вывод

8 Контрольные вопросы

8.1 Что такое AXAML?

8.2 Что такое Avalonia UI?

8.3 Как создать проект Avalonia UI?

8.4 Какие контейнеры компоновки используются в приложениях Avalonia UI

Практическая работа №20
Использование элементов управления для ввода данных

1 Цель работы

1.1 Научиться применять элементы управления Avalonia UI для ввода и вывода данных.

2 Литература

2.1 Avalonia documentation – Текст : электронный // AvaloniaUI, 2024. – URL: https://docs.avaloniaui.net/

3 Подготовка к работе

3.1 Повторить теоретический материал (см.п.2).

3.2 Изучить описание лабораторной работы.

4 Основное оборудование

4.1 Персональный компьютер.

5 Задание

5.1 Создайте новую страницу ControlsView и класс ControlsViewModel, настройте взаимодействие между ними.

5.2 Применение TextBlock

5.2.1 Расположите на странице TextBlock

5.2.2 Внутри TextBlock разместите 4 элемента Run с текстом. Для каждого из них установите различный цвет шрифта.

5.2.3 Между 2 и 3 Run расположите элемент LineBreak для переноса строки

5.2.4 Добавьте в TextBlock элемент Span, переместите внутрь его первый и второй Run. Для Span укажите другой размер шрифта и цвет фона.

5.2.5 Используя элементы Bold, Italic, Underline измените начертание фрагментов текста в TextBlock.

5.2.6 Добавьте в TextBlock изображение при помощи InlineUIContainer

```
<InlineUIContainer BaselineAlignment="Baseline">
    <Image
        Width="32"
        Height="32"
        VerticalAlignment="Top"
        Source="/Assets/avalonia-logo.ico" />
</InlineUIContainer>
```

5.2.7 Добавьте в TextBlock кнопку при помощи InlineUIContainer

5.3 Применение TextBox

5.3.1 Расположить на странице 2 TextBox для логина и пароля.

5.3.2 Указать для них Watermark для отображения подписи для поля ввода

5.3.3 Для поля ввода пароля указать PasswordChar

5.3.4 Добавить еще несколько TextBox для ввода Email и числового кода, указать для них различные значения TextInputOptions.ContentType

5.4 Применение CheckBox

5.4.1 Расположить на странице CheckBox. Протестировать его работу.

5.5 Применение RadioButton

5.5.1 Расположить на странице 4 RadioButton. Для каждой пары RadioButton указать различное значение свойства GroupName.

5.5.2 Протестировать работу RadioButton.

5.6 Применение CalendarDatePicker

5.6.1 Расположить на странице CalendarDatePicker для выбора даты рождения.

5.6.2 Добавить на страницу метку, выводящую значение возраста человека на основе выбранной даты. Для этого создайте вычисляемое свойство Age, которое вы будете оповещать об изменении основного свойства selectedDate

```
[ObservableProperty]
[NotifyPropertyChangedFor(nameof(Age))]
private DateTime selectedDate;

public int Age => //вычисление возраста
```

5.7 Применение ComboBox

5.7.1 Расположить на странице ComboBox и TextBlock

5.7.2 Создать в ControlsViewModel свойство Languages, содержащее список языков программирования, и поле selectedLanguage, помеченное атрибутом ObservableProperty.

5.7.3 Осуществить привязку свойств к ComboBox для выбора языка, и TextBlock для отображения выбранного. Протестировать их работу

5.8 Применение Slider

5.8.1 Расположить на странице Slider и Rectange, залитый цветом

5.8.2 В зависимости от значения слайдера необходимо изменять прозрачность Rectange.

5.9 Применение ToggleSwitch

5.9.1 Расположить на странице ToggleSwitch, установить для него значение OnContent и OffContent, для отображения текста Включено/Выключено

5.10 Валидация свойств

5.10.1 Создайте новую страницу Avalonia (UserControl) – RegistrationView. На странице расположите следующие элементы:

Поле ввода логина

Поле ввода пароля

Поле ввода подтверждения пароля

Поле ввода email

Поле ввода номера телефона

5.10.2 Создайте класс UserCreditionals, унаследованный от ObservableValidator

5.10.3 Определите в классе поля для всех необходимых свойств привязки:

```
[ObservableProperty]
[NotifyDataErrorInfo]
private string name;
```

NotifyDataErrorInfo используется для оповещения о изменении ошибок валидации при изменении значения свойства.

5.10.4 Создайте новый ViewModel – RegistrationViewModel. Определите в ней свойство для привязки типа UserCreditionals.

5.10.5 Используя атрибуты валидации, установите в UserCreditionals следующие ограничения:

Логин может содержать только латинские буквы, цифры и нижнее подчеркивание, длина логина – не менее 3 символов и не более 20

Пароль должен быть длиной не менее 8 символов, содержать латинские буквы верхнего и нижнего регистра, спецсимволы

Email должен содержать корректный email, а номер телефона - корректный номер.

Все поля должны быть заполнены.

5.10.6 Проверить работу валидаторов полей ввода.

6 Порядок выполнения работы

6.1 Выполнить все задания из п.5.

6.2 Ответить на контрольные вопросы.

7 Содержание отчета

7.1 Титульный лист

7.2 Цель работы

7.3 Ответы на контрольные вопросы

7.4 Вывод

8 Контрольные вопросы

8.1 Какой элемент управления используется для ввода текстовых данных?

8.2 Какие элементы управления используются для выбора одного или нескольких вариантов из списка?

8.3 От какого класса должны наследоваться классы с поддержкой валидации?

Практическая работа №21
Применение страничной навигации

1 Цель работы

1.1 Научиться выполнять навигацию по страницам приложения Avalonia UI.

2 Литература

2.1 Avalonia documentation – Текст : электронный // AvaloniaUI, 2024. – URL: https://docs.avaloniaui.net/

3 Подготовка к работе

3.1 Повторить теоретический материал (см.п.2).

3.2 Изучить описание лабораторной работы.

4 Основное оборудование

4.1 Персональный компьютер.

5 Задание

5.1 ViewLocator

5.1.1 Создайте в корне проекта класс со следующим содержимым:

```
public class ViewLocator : IDataTemplate
{
    public bool SupportsRecycling => false;
    public Control Build(object data)
    {
        var name =
        data.GetType().FullName
            .Replace("ViewModel", "View");
        var type = Type.GetType(name);
        if (type != null)
        {
            return (Control)Activator
                .CreateInstance(type);
        }
        else
        {
            return new TextBlock
                { Text = "Not Found: " + name };
        }
    }
    public bool Match(object data)
    {
        return data is ViewModelBase;
    }
```

```
}
```

5.1.2 Добавить в App.axaml следующие строчки:

```
<Application
    xmlns:local="using:ИмяПроекта"
    …
    />

<Application.DataTemplates>
    <local:ViewLocator/>
</Application.DataTemplates>
```

ViewLocator позволяет автоматически создавать объект View для объекта ViewModel в приложении

5.2 Переходы между страницами

5.2.1 В MainViewModel создайте ObservableProperty поле currentPage типа ViewModelBase

5.2.2 Создайте 2 страницы RegistrationView и AuthorizationView, расположите на них соответствующие элементы управления

5.2.3 В MainWindow создайте элемент управления ContentControl и выполните привязку его свойства Content к значению CurrentPage

5.2.4 Создайте в MainWindow две кнопки, создайте и привяжите к ним команды которые будут присваивать CurrentPage соответствующее значение (RegistrationViewModel или AuthorizationViewModel)

5.2.5 Протестируйте преходы между страницами

5.3 Сервис навигиции

5.3.1 Создайте новый класс NavigationService

5.3.2 В классе определите навигационный стек в виде Stack<ViewModelBase> и 2 метода для управления этим стеком: универсальный NavigateTo<T>, где T – класс-наследник ViewModelBase и GoBack

```
public void NavigateTo<T>(T viewModel, Action<T>? action
= null) where T : ViewModelBase
{
    //TODO
}

public void GoBack(int steps = 1)
{
    //TODO
}
```

5.3.3 При выполнении навигации в метод может передаваться делегат Action<T>, который позволит произвести настройки для страницы (установить значения некоторых свойств). Для вызова делегата необходимо выполнить

```
action?.Invoke(viewModel);
```

5.3.4 Добавьте экземпляр сервиса в качестве статического публичного члена в MainViewModel

5.3.5 Измените реализацию навигации в приложении с использованием созданного сервиса.

6 Порядок выполнения работы

6.1 Выполнить все задания из п.5.

6.2 Ответить на контрольные вопросы.

7 Содержание отчета

7.1 Титульный лист

7.2 Цель работы

7.3 Ответы на контрольные вопросы

7.4 Вывод

8 Контрольные вопросы

8.1 Для чего применяется ViewLocator?

8.2 Как реализовать переход между страницами, обратный переход?

8.3 Как передать данные между страницами?